

Contents

Unified Dynamic Framework + QD↔Dynamic Cycle Wiring — Plan v1 (draft)	2
1. Constraints (what cannot move, what is fair game)	2
1.1 Hard “do not edit” zone	2
1.2 Stable interfaces (must keep byte-equivalent behavior)	2
1.3 Editable surface	2
1.4 Numerical / physics constraints	3
2. Current-state recap (for quickly orienting round-2 readers)	3
2.1 Dynamic side: what works, what hurts	3
2.2 QD side: what works, what’s missing for cycling	3
2.3 Where prior plans intersect this one	4
3. Part A: Unified Dynamic Framework	4
3.1 The three axes of variability	4
3.2 Proposed abstraction surface	4
3.3 Setup unification	7
3.4 Driver unification	8
3.5 Phase plan for the unified-framework refactor	8
4. Part B: QD↔Dynamic Cycle Wiring	10
4.1 Scope of v1 cycling	10
4.2 Naming, geometry, and mesh	10
4.3 State vector reconciliation	11
4.4 QD → Dynamic handoff (one-time per event)	11
4.5 Dynamic → QD handoff (one-time per event)	12
4.6 Trigger logic	13
4.7 Phase plan for cycle wiring	13
5. Testing strategy	16
6. Risk register (preliminary)	16
7. What’s NOT in this plan	17
8. Files summary (what gets touched in v1)	17
New (Part A)	17
New (Part B)	18
Modified (Part A)	18
Modified (Part B)	18
Removed (after A.7 only, gated on byte-for-byte goldens)	18
9. Success criteria (exit conditions for v1 implementation)	19
10. Approximate effort estimate	19
11. Open questions for round 2 (please weigh in)	19
11.1 Template vs virtual dispatch for the strategies	19
11.2 Mesh strategy for hybrid mode	19
11.3 What’s a realistic round-trip tolerance?	20
11.4 TOML config: now or later?	20
11.5 Dynamic rupture pure-mode driver: keep or sunset?	20
11.6 Ordering of Part A and Part B	20
11.7 Other QD-state variables besides ψ	20
12. Round-2 inputs we’d like	20
13. Round-1 Adversarial Review (added 2026-04-29)	21
Critical issues (must resolve before implementation)	21
Moderate issues	24
Low / quality issues	28
Summary of required v2 actions	28

Unified Dynamic Framework + QD↔Dynamic Cycle Wiring — Plan v1 (draft)

Date: 2026-04-27 Status: Round-1 draft. Intended as a starting point; expect revisions in v2/v3 after the user weighs in on the design choices flagged in §11. Scope: Two coupled refactors of the SEAS-MFEM codebase: 1. Unified dynamic-rupture framework — collapse the per-benchmark trio (TPV102 / TPV104 / TPV205 each carry their own *_setup.hpp, *_substep_iterator.{hpp,cpp}, *_friction_solver.hpp, *_nucleation.hpp plus a near-duplicate driver) into a parametrized framework with pluggable friction law, nucleation strategy, and state evolution. 2. QD↔Dynamic cycle wiring — wire the existing BP5 quasi-dynamic operator (rate-state aging, vector slip in the dip/strike frame) to the existing dynamic operator so that a single run can simulate the long aseismic phase quasi-dynamically and switch to fully dynamic during rupture, then switch back.

Predecessor documents (in this folder): - dynamic_rupture_plan_v4.md (2026-04-12) — wrote the dynamic rupture solver from the ground up. Phase 5 (“BP5 QD→Dynamic Transfer”) sketches the hybrid; pre-dates the now-shipped TPV102/104/205. The handoff math, CG↔DG projection, equilibrium correction, and SEASHybridOperator switching ideas there are still relevant and consumed below where applicable. - bp5_refactoring_plan_v4.md (2026-04-09) — Section 12 (“Future Architecture”) proposed mode = "qd" | "dynamic" | "hybrid" as a TOML-level driver mode. The current plan adopts that framing. - tpv205_lsw_native_fields_plan_2026-04-27.md — the live D0FData LSW-field design that permits the friction-law dispatch via enum + native-field slots. The unified-framework design here generalizes that approach. - interface_prep_plan_03012026.md, mfem_seas_system_updateplan_02282026.md — earlier “make BP5 possible” prep plans, mostly historical now.

1. Constraints (what cannot move, what is fair game)

1.1 Hard “do not edit” zone

Per saved feedback feedback_dynamic_folder_editable_for_tpv104 and feedback_tpv102_bp5_no_shared_edit:
- No edits to bp5/, bp1/, bp2/, domain/, fault/, solver/, friction/dieterich_ruina.hpp, friction/state_evolution.hpp, friction/slip_law_srw_psi.hpp, friction/friction_coeff_stable.hpp, or dynamic/tpv102_setup_total.hpp.
- This means the unified framework must be additive on the dynamic side: it adds new files in dynamic/ (or a new sub-namespace) that can coexist with the per-benchmark trio until cutover.

1.2 Stable interfaces (must keep byte-equivalent behavior)

- FaultFaceFlux::{ComputeTrialTraction, ComputeStageState, BuildImposedState, WriteBackState, EvaluateADER[_Total][_LSW]} — TPV102/104/205 all depend on these byte-for-byte (per tpv205_lsw_native_fields_plan Phase 1).
- WaveOperator<MeshType>::{Mult, AdvanceADER, ComputeADERTimeIntegrated, ComputeADERSubStepStates, EvaluateBulkAtFaultQPsCanonical, SetFaultFrictionLaw, SetSubStepFaultImposedStates} — same.
- D0FData field offsets (only append new fields).
- SEASQuasiDynamicOperator<MeshType, DomainOpType> template signature.
- RateStateFaultOperator<MeshType, NumSlipComp> state layout [s1, s2, ψ] per node for NumSlipComp=2.
- DormandPrinceRK45 interface (used by QD).
- BP5Params field set (no edits to config/bp5_params.hpp).

1.3 Editable surface

- dynamic/ everything except tpv102_setup_total.hpp (new files welcome, the per-benchmark trio is fair game once the unified framework matches each benchmark byte-for-byte).
- drivers/{tpv102, tpv104, tpv205}_driver.cpp (the entire ~2200-line driver each).
- drivers/seas_driver.cpp (BP5 QD driver — additive only; do not break BP5 production).

- New top-level driver(s) for unified dynamic and unified hybrid.
- config/ for new config struct(s) covering dynamic + hybrid modes.

1.4 Numerical / physics constraints

- Fault-local frame: $\text{tangent1} = \text{dip}$, $\text{tangent2} = \text{strike}$ (Tandem/BP5 convention; project-wide per `miniapps/seas/CLAUDE.md`). Any new code must respect this; no re-invention of frame conventions.
- State variable ψ continuity across QD $\leftrightarrow$ dynamic handoff is mandatory — both sides use the regularized $f(V, \psi, a) = a \cdot \text{asinh}[(V/2V_0) \cdot \exp(\psi/a)]$ formulation with the aging-law ODE $d\psi/dt = (b \cdot V_0/D_c) \cdot [\exp((f_0 - \psi)/b) - V/V_0]$. This is the only friction law combination for which the cycle is well-defined out of the box. Slip-law-SRW (TPV104) and LSW (TPV205) cycling are explicitly out of scope for v1; see §11 Open Questions.
- Sign conventions (slip-rate parallel to traction; pre-stress parallel to initial velocity; $\sigma_n > 0 = \text{compression}$) are load-bearing and must be preserved at every handoff boundary.

2. Current-state recap (for quickly orienting round-2 readers)

2.1 Dynamic side: what works, what hurts

Works (verified via SCEC benchmarks): - TPV102: regularized R&S with aging-law ψ evolution, time-varying Gaussian nucleation ($\Delta\tau = 25$ MPa, $R = 3$ km). - TPV104: regularized R&S with slip-law + strong slip-rate weakening (V_w side-channel), time-varying Gaussian nucleation ($\Delta\tau = 45$ MPa). - TPV205: linear slip-weakening (LSW), no state variable, three static pre-stress patches + strength-barrier outside the rupture area (no time-varying nucleation).

Shared, clean (no TPV-specific branches, the refactor leaves these alone): - `dynamic/wave_operator.{hpp,inl}` — DG wave op + ADER predictor; one dispatch site if (`fault_friction_law_ == LSW`) only. - `dynamic/fault_face_flux.{hpp,cpp}` — fault Riemann + imposed-state pipeline; LSW-vs-R&S selection by which `D0FData` fields are populated, not by branches. - `dynamic/godunov_flux.{hpp,cpp}`, `dynamic/precomputed_face_fluxes.{hpp,cpp}`, `dynamic/wave_state.hpp`, `dynamic/seas_dynamic_operator.hpp`, `dynamic/pml_layer.{hpp,cpp}`, `dynamic/d4_tet_mesh.hpp`, `dynamic/fault_locality_partition.hpp`, `dynamic/shared_fault_key.hpp` — all TPV-agnostic. - `dynamic/friction_solver.{hpp,cpp}` — generic Brent/Newton/Hybrid solver for the R&S balance; LSW does not flow through this (it goes through `tpv205_friction.hpp::SolveLSW_TPV205`).

Hurts (the duplication that motivates this plan): - `dynamic/tpv1XX_setup.hpp` — 504 / 552 / 512 lines, ~85 % shared structure (mesh probe lookup, station writer, MPI broadcast, $t = 0$ fault-state fill). 15 % differs in friction-law fields, ψ initialization anchor, and V_w side-channel (TPV104 only). - `dynamic/tpv1XX_substep_iterator.{hpp,cpp}` — same per-substep skeleton: nucleation increment $\rightarrow$ trial traction $\rightarrow$ friction solve $\rightarrow$ state update $\rightarrow$ imposed-state build $\rightarrow$ accumulate. The differences are exactly the three axes named in §3.1. - `dynamic/tpv1XX_friction_solver.hpp` — TPV102 is a 1-line alias forwarding to TPV104's `SolveSlipRateNewtonStable_TPV104`. TPV205 is a closed-form algebraic solve. The Newton wrapper itself is fine; the per-benchmark wrappers are noise. - `dynamic/tpv102_nucleation.hpp` and `dynamic/tpv104_nucleation.hpp` — same Gaussian smoothStep formula; differ in the timing semantics (overwrite vs accumulate per substep) and in ($\Delta\tau$, R , hypocenter), all of which are parameters not code. - `drivers/tpv1XX_driver.cpp` — ~2200 lines each, ~70 % boilerplate (mesh load, MPI init, ParaView wiring, time-loop skeleton, station I/O), ~30 % per-benchmark wiring (`InitializeFaultD0Fs_TPV1XX`, iterator construction, station writer class, output column ordering, optional V_w side-channel).

2.2 QD side: what works, what's missing for cycling

Works: - BP5 QD: vector slip [s_{dip} , s_{strike} , ψ] per fault DOF, regularized R&S aging-law, RK45 with PI controller + V-guard, MUMPS-direct elasticity solve per RHS evaluation, optional elastic σ_n . - 4-phase

initialization (Prelinit → domain solve → Init from equilibrium → Verify) producing equilibrium error < 1e-6. - Earthquake-detection thresholds in `drivers/seas_driver.cpp:606–615` ($V_{\max} > 1e-3$ enters seismic, $V_{\max} < 1e-6$ exits).

Missing for cycling: - No code path that handles “earthquake detected” beyond bumping `eq_count` and adapting output frequency. The thresholds are hard-coded in the driver; nothing in `solver/` or `dynamic/` consumes them as a regime switch. - No state-conversion utilities: QD state is `[s_dip, s_strike,  $\psi$ ]` per DOF; dynamic state is the bulk Q (`NUM_STATE = 9` per DG dof) plus a separate `DOFData` array. The conversion is one-way coded in `seas_driver.cpp` for initial setup but not parameterized for “switch in the middle of a run.” - No bulk-velocity field tracked in QD (quasi-static); a dynamic kick-off needs an initial $v(x)$ decision (§5.2 of `dynamic_rupture_plan_v4.md` proposes $v = 0 + 5$ -step damped ramp on nucleation; that proposal is endorsed below). - No “rupture has stopped” detection in `WaveOperator` or any dynamic driver. - No `SEASHybridOperator` class; `dynamic_rupture_plan_v4 §12.1` sketches it but it was never built.

2.3 Where prior plans intersect this one

- `dynamic_rupture_plan_v4.md §5` (“BP5 QD→Dynamic Transfer”) is the closest existing thinking. It pre-dates the TPV implementations, so its assumption that the dynamic driver is greenfield is wrong now — the dynamic driver is already three drivers, and this plan handles that. Its math (CG→DG projection, fault-tip taper, equilibrium-correction sweep, FD→QD 5-step re-init) is reused below.
- `bp5_refactoring_plan_v4.md §12.1` (`mode = "qd" | "dynamic" | "hybrid"` config) is the unifying surface. v1 of this plan adopts the same surface but stays config-format-agnostic for now (TOML migration is a separate effort; see §11).

3. Part A: Unified Dynamic Framework

3.1 The three axes of variability

The dynamic-side analysis shows that TPV102 / TPV104 / TPV205 differ along exactly three axes plus driver-level cosmetics:

Axis	TPV102	TPV104	TPV205	Pluggability
Friction law ($\tau \leftrightarrow V$ relation)	Reg. R&S, Newton	Reg. R&S, Newton (same wrapper as TPV102)	LSW, closed-form	Strategy / template parameter
State evolution (ψ ODE)	Aging law	Slip law + SRW (V_w side-channel)	None	Strategy / template parameter
Nucleation (per-step pre-stress perturbation)	Gaussian smoothStep, overwrite τ_{2_0} per macro-step, $\Delta\tau = 25$ MPa, $R = 3$ km	Gaussian smoothStep, accumulate $\Delta\tau_{\text{nuc}}$ per substep, $\Delta\tau = 45$ MPa, $R = 3$ km	None (static pre-stress at $t = 0$)	Strategy with parameter struct

Driver-level differences (initial-state filler, output column order, optional side-channel arrays, station-writer columns, CLI flag set) are addressable by templating + a small per-benchmark traits struct.

3.2 Proposed abstraction surface

Decision needed (§11.1): template-vs-virtual. The trade-off is: - Templates (compile-time): zero runtime overhead, native MFEM `Vector/real_t` types flow naturally, but each benchmark instantiates its own substep-iterator object code ($\approx \times 3$ binary size for the iterator translation unit only — minor for a research code). -

Virtual dispatch: single binary, easier to load benchmarks from config strings, but adds an indirect call per fault-QP per substep (could be measurable since the inner loop is hot).

This plan assumes templates for v1. If v2 wants virtual dispatch, the strategy interfaces below are easy to virtualize because they already package the per-call data.

3.2.1 FrictionLaw strategy (header `dynamic/framework/friction_law_strategy.hpp`)

```
namespace mfem::seas::dyn {

// Encapsulates  $\tau \rightarrow V$  solve and (optionally)  $\sigma_n \rightarrow V$  correction.
// Implementations stored in DOFData via native fields; this struct is stateless.
template <class Tag>
struct FrictionLawStrategy;

// Specialization for regularized rate-and-state, Newton solve.
struct RateStateNewtonTag {};
template <>
struct FrictionLawStrategy<RateStateNewtonTag> {
    static EvalStageState SolveStage(
        const DOFData& d,
        const real_t tau1_trial, const real_t tau2_trial,
        const real_t sigma_n_trial,
        const real_t dt_sub);
    // Returns: {V, V1, V2, tau1_corr, tau2_corr, sigma_n_corr, converged?}
    // Calls FrictionSolver::Solve under the hood.
};

// Specialization for closed-form LSW.
struct LSWClosedFormTag {};
template <>
struct FrictionLawStrategy<LSWClosedFormTag> {
    static EvalStageState SolveStage(
        const DOFData& d,
        const real_t tau1_trial, const real_t tau2_trial,
        const real_t sigma_n_trial,
        const real_t dt_sub);
    // Calls SolveLSW_TPV205. Reads d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c.
};

} // namespace
```

3.2.2 StateEvolution strategy (header `dynamic/framework/state_evolution_strategy.hpp`)

```
template <class Tag>
struct StateEvolutionStrategy;

struct AgingLawPsiTag {};
template <>
struct StateEvolutionStrategy<AgingLawPsiTag> {
    static void Advance(DOFData& d, real_t V_abs, real_t dt_sub);
    // Calls UpdateStateAnalytic from friction/state_evolution.hpp
};

struct SlipLawSRWPsiTag {};
```

```

template <>
struct StateEvolutionStrategy<SlipLawSRWPsiTag> {
    static void Advance(DOFData& d, real_t V_abs, real_t dt_sub);
    // Calls UpdateStateAnalyticSlipLawSRW; reads d.V_w
};

struct NoStateTag {};
template <>
struct StateEvolutionStrategy<NoStateTag> {
    static void Advance(DOFData&, real_t, real_t) {} // no-op for LSW
};

```

3.2.3 Nucleation strategy (header `dynamic/framework/nucleation_strategy.hpp`)

```

struct NucleationParams {
    real_t delta_tau; // peak amplitude (Pa)
    real_t T_nuc; // duration of smoothStep (s)
    real_t R_nuc; // patch radius (m)
    real_t hypocenter_x;
    real_t hypocenter_z;
};

// Strategy is responsible for: (a) when to apply (per macro-step or per substep),
// (b) overwrite or accumulate semantics, (c) static or time-varying.
template <class Tag>
struct NucleationStrategy;

struct GaussianSmoothStepOverwriteTag {}; // TPV102
struct GaussianSmoothStepAccumulateTag {}; // TPV104
struct StaticPreStressTag {}; // TPV205 (no-op at t > 0)

template <class Tag>
struct NucleationStrategy {
    static void Apply(
        std::vector<DOFData>& dof_data,
        const Vector& fault_x, const Vector& fault_z, // 2D fault coords
        const NucleationParams& params,
        real_t t_macro,
        real_t dt_sub,
        int sub_step_index);
};

```

The key payoff: the substep iterator becomes one templated class:

```

// dynamic/framework/substep_iterator.hpp
template <class FrictionTag, class StateTag, class NucleationTag>
class SubStepIterator {
    FaultFaceFlux& flux_;
    NucleationParams nuc_params_;
public:
    void Advance(const Vector& I_plus_in, const Vector& I_minus_in,
                Vector& I_plus_imp_out, Vector& I_minus_imp_out,
                real_t t_macro, real_t dt_macro, int ader_order);
};

```

Each benchmark gets a typedef:

```
// dynamic/framework/benchmarks.hpp
using TPV102Iterator = SubStepIterator<RateStateNewtonTag, AgingLawPsiTag, GaussianSmoothStepOverwriteTag>
using TPV104Iterator = SubStepIterator<RateStateNewtonTag, SlipLawSRWPsiTag, GaussianSmoothStepAccumulateTag>
using TPV205Iterator = SubStepIterator<LSWClosedFormTag, NoStateTag, StaticPreStressTag>;
```

3.3 Setup unification

Per-benchmark setup files become trait-style headers that fill a shared BenchmarkConfig struct:

```
// dynamic/framework/benchmark_config.hpp
struct BenchmarkConfig {
    // Fault geometry / mesh
    std::string mesh_path;
    real_t fault_z_top, fault_z_bottom;    // depth bounds

    // Material
    real_t lambda, mu, rho;

    // Pre-stress (BP5 frame: t1 = dip, t2 = strike)
    real_t sigma_n_0;
    real_t tau1_0_default;                // 0 for pure strike-slip
    std::function<real_t(real_t x, real_t z)> tau2_0_field;

    // Friction-law parameters (rate-state path)
    std::optional<real_t> V0, f0, b;        // empty when LSW
    std::function<real_t(real_t x, real_t z)> a_field;        // R&S
    std::function<real_t(real_t x, real_t z)> Dc_field;        // R&S
    std::function<real_t(real_t x, real_t z)> Vw_field;        // R&S+SRW (TPV104)

    // Friction-law parameters (LSW path)
    std::function<real_t(real_t x, real_t z)> lsw_mu_s_field;
    std::function<real_t(real_t x, real_t z)> lsw_mu_d_field;
    std::function<real_t(real_t x, real_t z)> lsw_d_c_field;

    // Initial slip rate
    real_t V_init;

    // Nucleation
    NucleationParams nuc;

    // Output config (column order, station coords, etc.)
    StationWriterConfig output;
};
```

The three setup files become factory functions:

```
// dynamic/framework/benchmarks/tpv102_config.hpp
BenchmarkConfig MakeTPV102Config(int argc, char* argv[]);

// dynamic/framework/benchmarks/tpv104_config.hpp
BenchmarkConfig MakeTPV104Config(int argc, char* argv[]);

// dynamic/framework/benchmarks/tpv205_config.hpp
BenchmarkConfig MakeTPV205Config(int argc, char* argv[]);
```

Each is ~50–100 lines and only encodes the parameter values + initial conditions. The shared filler —

InitializeFaultDOFsFromConfig(BenchmarkConfig&, std::vector<DOFData>&) — moves to the framework.

3.4 Driver unification

Three drivers → one binary. Two options:

- Option A (single binary, `--benchmark` flag): `seas_dynamic_driver --benchmark tpv102 [...]`. CLI flag dispatches to `MakeTPV1XXConfig(argc, argv)` then runs the templated time loop.
- Option B (single binary, TOML config): `seas_dynamic_driver --input tpv102.toml`. Fully matches the `bp5_refactoring_plan_v4.md` §12 vision but adds TOML-parsing dependency.

Recommendation: Option A for v1, deferring TOML to its own plan (see §11.4). It cuts code volume immediately without taking on a parser dependency.

The unified driver `drivers/seas_dynamic_driver.cpp` skeleton:

```
int main(int argc, char* argv[]) {
    MPIContext mpi(&argc, &argv);
    const std::string bench = ParseBenchmarkFlag(argc, argv);

    BenchmarkConfig cfg = (bench == "tpv102") ? MakeTPV102Config(argc, argv)
        : (bench == "tpv104") ? MakeTPV104Config(argc, argv)
        : (bench == "tpv205") ? MakeTPV205Config(argc, argv)
        : (Abort("unknown benchmark"), BenchmarkConfig{});

    // Construct mesh, wave_op, fault_flux from cfg (shared code; ~400 lines today are duplicated).
    auto [mesh, wave_op, fault_flux] = BuildDomain(cfg, mpi);

    // Construct iterator via type dispatch (one-liner for each benchmark).
    if (bench == "tpv102") {
        TPV102Iterator iter(*fault_flux, cfg.nuc);
        RunTimeLoop(iter, cfg, mpi); // shared time loop
    } else if (bench == "tpv104") {
        TPV104Iterator iter(*fault_flux, cfg.nuc);
        RunTimeLoop(iter, cfg, mpi);
    } else {
        TPV205Iterator iter(*fault_flux, cfg.nuc);
        RunTimeLoop(iter, cfg, mpi);
    }
}
```

`RunTimeLoop` is a function template that takes `auto& iterator` (C++20 abbreviated template) and contains the ~600-line shared time-loop / output / probe / event-tracking code currently duplicated three times.

3.5 Phase plan for the unified-framework refactor

Phase	Goal	Files created	Files modified	Files removed	Acceptance
A.0	Pin down baseline	(regression script)	none	none	All three TPV drivers reproduce their golden outputs to byte-for-byte (or to within an explicit numerical tolerance documented in regression.toml)
A.1	Strategy interfaces, no callers	dynamic/framework/ none (templates with empty specializations stubbed to call existing per-benchmark functions)	none friction_law, none none_evolution	none	Compiles. New headers have no callers.
A.2	Templated SubStepIterator + per-benchmark typedefs	dynamic/framework/ none substep_iterator.hpp, dynamic/framework/benchmarks.hpp	none	none	Compiles. New iterator class has no callers.
A.3	Migrate TPV102 to framework	dynamic/framework/ dr/bench/tpv102.hpp (use TPV102Iterator + new config)	dr/bench/tpv102.hpp	dr/bench/tpv102_config.hpp	TPV102 golden reproduced to byte-for-byte (or documented tolerance $\leq 1e-12$). Old dynamic/tpv102_*.hpp stays as fallback.
A.4	Migrate TPV104 to framework	dynamic/framework/ dr/bench/tpv104.hpp	dr/bench/tpv104.hpp	dr/bench/tpv104_config.hpp	TPV104 golden reproduced. Old files stay.
A.5	Migrate TPV205 to framework	dynamic/framework/ dr/bench/tpv205.hpp	dr/bench/tpv205.hpp	dr/bench/tpv205_config.hpp	TPV205 golden reproduced (Frontera np > 1 since LSW is sensitive to MPI rank boundaries per tpv205_lsw_native_fields_platform Phase 4).
A.6	Unify driver	drivers/seas_dyn (build system: cpp, none) dynamic/framework/ add unified _loop.hpp target, keep old targets as _legacy)	dynamic/framework/add unified _loop.hpp	dynamic/framework/add unified _loop.hpp	All three benchmarks reproduce golden via single binary.

Phase	Goal	Files created	Files modified	Files removed	Acceptance
A.7	Remove duplication	none	(build system: drop_legacy targets)	dynamic/tpv1XX_setup_tot (preserve tpv102_setup_tot reduction) dynamic/tpv1XX_delet (now_legacy)	Code size win: ~6000 lines deleted (~2500 added). All golden still reproduce.

Each phase is independently mergeable: the codebase compiles and goldens reproduce after each phase. Phases A.3–A.5 can be parallelized once A.1–A.2 are merged.

4. Part B: QD↔Dynamic Cycle Wiring

This is the bigger piece. v1 sketches the architecture; many details are deferred to v2 once §11 questions are answered.

4.1 Scope of v1 cycling

In scope: BP5 QD ↔ TPV102-style dynamic. Both use: - Vector slip in dip/strike frame ($\text{NumSlipComp} = 2$). - Regularized R&S friction with the Tandem $f(V, \psi, a) = a \cdot \text{asinh}[\dots]$ formulation. - Aging-law ψ evolution. - 3D geometry, planar vertical fault.

This pairing is the only one for which ψ is directly transferable without re-derivation. The friction law and state evolution match exactly.

Out of scope for v1 (deferred to v2/v3): - TPV104-style cycling (slip-law + SRW): would need QD-side support for slip-law ψ ODE and V_w field. Doable, but not now. - TPV205-style cycling (LSW): no ψ at all; the QD↔dynamic handoff has no state to carry. Conceptually different (would require tracking accumulated slip δ as the “memory” variable). Defer. - Fully dynamic-only mode (no QD warmup) — already exists via the unified dynamic driver from Part A.

4.2 Naming, geometry, and mesh

The unified hybrid will need a single 3D mesh annotated for both phases: - Dirichlet far-field surfaces (loading via $u = \pm V_p \cdot t / 2$). - Free-surface top. - Fault surface (Gmsh Physical Surface, BP5 convention: tag 3). - Absorbing/PML region (only relevant during dynamic phase; QD ignores).

BP5 mesh bp5/mesh/bp5_*.msh is already 3D and tagged (bp5_tandem_exact.msh is the production reference). For TPV102, the existing dynamic mesh is similar in topology (vertical $y = 0$ fault) but uses a Cartesian box from dynamic/d4_tet_mesh.hpp-derived generators or the per-driver inline generator. For the cycle, we will need a single mesh that: - Is 3D, fully tagged for both BC types. - Has fault tagged with BP5’s convention. - Is large enough that absorbing BC reflections during a coseismic event do not contaminate post-event QD ramp-up (this is the artifact concern dynamic_rupture_plan v4 §5.2 raised).

Decision needed (§11.2): mesh strategy. Options: - Use BP5 production mesh for all hybrid runs. Pro: existing benchmark. Con: way too big for short TPV-style verification runs. - Build a “hybrid verification” mesh (smaller, e.g. $30 \times 30 \times 30$ km box, vertical fault $y = 0$, BP5-style tags). Pro: fast iteration. Con: new artifact.

4.3 State vector reconciliation

The QD time-stepper integrates a state of size $3 N_{\text{fault_dofs}}$ (s_{dip} , s_{strike} , ψ per DOF). The dynamic time-stepper integrates a bulk state of size $9 N_{\text{dg_dofs}}$ (the velocity-stress vector Q) plus a separate `std::vector<DOFData>` indexed by fault-face quadrature points.

These are not compatible objects. The `SEASHybridOperator` will need to hold both:

```
// solver/seas_hybrid_operator.hpp (NEW)
template <class MeshType>
class SEASHybridOperator : public TimeDependentOperator {
    SEASQuasiDynamicOperator<MeshType, ElasticityDomainOperator<MeshType>>* qd_op_;
    SEASDynamicOperator<MeshType>* dyn_op_;           // wraps WaveOperator + FaultFaceFlux

    enum class Regime { QuasiDynamic, Dynamic };
    Regime current_;

    // Storage owned by the hybrid op:
    Vector qd_state_;      // 3*N_fault - slip,  $\psi$ 
    Vector dyn_state_;     // 9*N_dg - bulk Q
    std::vector<DOFData> dyn_dof_data_;

    // Trigger thresholds:
    real_t V_activate_;    // QD  $\rightarrow$  dynamic, default 1e-3 m/s
    real_t V_deactivate_;  // dynamic  $\rightarrow$  QD, default 1e-6 m/s
    real_t exit_grace_t_;  // additional wait after  $V < V_{\text{deactivate}}$  (s), default 0.0

public:
    void Mult(const Vector& y, Vector& dy_dt) const override; // dispatches by regime
    void Step(real_t& t, real_t& dt); // owns time-stepping + regime switching
};
```

Critical subtlety: the hybrid operator can't inherit `TimeDependentOperator` cleanly because the size of its state vector changes when the regime switches ($3 N_{\text{fault}}$ vs $9 N_{\text{dg}}$). The cleanest approach is: - The hybrid op owns its own time-stepping loop (does not delegate to RK45 or the wave RK4 directly). - `Step()` advances internal state by one logical macro-step using the active regime's stepper, then checks the trigger.

This rules out exposing the hybrid op as a `TimeDependentOperator` to outside callers, but that's fine — the driver calls `hybrid_op.Step(t, dt)` in a loop.

4.4 QD $\rightarrow$ Dynamic handoff (one-time per event)

At the moment QD's $V_{\text{max}} > V_{\text{activate}}$:

1. Persist QD state. Save (s_{dip} , s_{strike} , ψ) per fault DOF — straight copy from `qd_state_`.
2. Compute current bulk displacement $u(x)$. This is the stationary displacement field for the current slip BC. The QD operator has just solved this (the elasticity solve inside the latest RHS evaluation). Re-solve once more to be sure we have it at the exact handoff t (not at the last RK stage).
3. Project u from CG H^1 space (QD) to DG L^2 space (dynamic). This is element-local (basis-change matrix per element), no MPI. Dynamic_rupture_plan v4 §5.2 estimates ~ 0.1 s for BP5 size. The basis-change matrix can be precomputed at startup. Output: bulk DG $u \rightarrow$ bulk DG $\epsilon(u) = \nabla^s u \rightarrow$ bulk DG $\sigma = C : \epsilon$ via `ConstitutiveModel::ComputeStress(grad_u)`.
4. Initialize bulk velocity v . Two options, both endorsed by literature:
 - Set $v = 0$ everywhere except the fault. On the fault, set $v(x) = \frac{1}{2} V(x)$ (each side moves at half the slip rate, away from each other). Apply a Gaussian taper away from the fault over $\sim 3\text{--}5$ elements

- to avoid a velocity discontinuity in the bulk DG space.
- Solve a “kick-off” problem that sets v consistent with σ via one explicit Euler step backwards in time (more accurate but adds plumbing). Defer this to $v2$.

5. Pack **dyn_state_**. Concatenate per-element σ (6 components) and v (3 components) into the 9-component Q at every DG dof. Standard DG pack.

6. Initialize **dyn_dof_data_**. Per fault DOF, populate D0FData from QD state:

```
data.psi = qd_psi(i)
data.slip1 = qd_s_dip(i)          // accumulated slip carries forward
data.slip2 = qd_s_strike(i)
data.V1, data.V2 = qd_V_dip(i), qd_V_strike(i) // from latest RHS
data.tau1_0, data.tau2_0 = qd_tau_dip(i), qd_tau_strike(i) // background pre-stress
data.sigma_n_0 = qd_sigma_n(i)
data.tau1_corr, data.tau2_corr, data.sigma_n_corr = 0
data.tau1_nuc, data.tau2_nuc, data.sigma_n_nuc = 0 // no nucleation in cycle mode
data.a, data.Dc = (from BenchmarkConfig)
data.Zp_plus, data.Zs_plus, data.Zp_minus, data.Zs_minus, data.eta_s = (from material)
```

7. **Equilibrium correction**. As proposed by dynamic_rupture_plan_v4 §5.2: run N_{settle} (~ 10) wave time steps with elevated PML damping to absorb the projection-induced transient before letting the rupture loose. The hybrid op exposes N_{settle} as a config parameter; default 10.

8. Switch regime. `current_ = Regime::Dynamic`. Subsequent `Step()` calls go through `dyn_op_`.

Open question (§11.3): whether step 7 (settling) is needed in the cycle context. In a cold start it absorbs the projection error; in a hot start during nucleation, it might wash out the nucleation-driving stress. Probably yes for $v1$, with a flag to disable.

4.5 Dynamic → QD handoff (one-time per event)

When `dynamic's  $V_{\text{max}} < V_{\text{deactivate}}$` for at least `exit_grace_t_` seconds:

1. Capture final fault state. Per DOF: `slip1_final = data.slip1`, `slip2_final = data.slip2`, `psi_final = data.psi`, `sigma_n_final = data.sigma_n_corr` (the running corrected normal stress, which equals σ_{n0} in pure strike-slip cases like TPV102 / BP5).
2. Optionally average bulk u over a wave-period to suppress oscillations. Defer to $v2$ — for $v1$, just pick `u(t_exit)`.
3. Repack QD state. Direct copy of `(slip_final, psi_final)` into `qd_state_`.
4. Discard bulk u, v . Quasi-dynamic does not track them; on the next `qd_op_.Mult()` call, the QD elasticity solve reconstructs u directly from slip BC.
5. Re-run 4-phase initialization? No — `psi_final` is already the right state, and re-running phase 3 would reset ψ to equilibrium with current pre-stress, destroying the dynamic-evolved healing. Instead, `dynamic_rupture_plan v4 §5.3` proposes a 5-step verification:
 1. Project $u_{\text{dyn}} \rightarrow u_{\text{qd}}$ via $L^2 \rightarrow H^1$ (volumetric L^2 -projection; one MUMPS solve).
 2. Compute traction on fault from u_{qd} .
 3. Verify friction equilibrium $|\tau_{\text{total}} - (\sigma_n \cdot f(V_{\text{qd}}, \psi_{\text{final}}, a) + \eta \cdot V_{\text{qd}})| / |\tau_{\text{total}}| < 1e-6$ with V_{qd} derived from the just-projected u (not from V_{final} , which is by definition $< V_{\text{deactivate}} \ll V_p$).
 4. If equilibrium fails, re-solve for V_{qd} given $(\tau, \psi_{\text{final}})$ via Brent — same as QD's phase 3 init.
 5. Set `qd_state_ = (slip_final,  $\psi_{\text{final}}$ )`, set `current_ = Regime::QuasiDynamic`, resume QD time-stepping with `dt = dt_qd_default` (e.g. 1e3 s).

For $v1$ the project- u step (5.1) is optional since QD will resolve u on its next RHS evaluation. The verification step (5.3) is mandatory and gates the regime switch.

4.6 Trigger logic

Inside SEASHybridOperator::Step():

```
void Step(real_t& t, real_t& dt) {
    if (current_ == Regime::QuasiDynamic) {
        qd_stepper_.Step(*qd_op_, qd_state_, t, dt);
        real_t V_max = ComputeVMax(qd_state_);
        MPI_Allreduce(MPI_IN_PLACE, &V_max, 1, MPI_DOUBLE, MPI_MAX, comm_);
        if (V_max > V_activate_) {
            HandoffQDtoDynamic();
            dt = dt_dyn_initial_; // e.g. CFL · h_min / cp
        }
    } else { // Dynamic
        dyn_stepper_.Step(*dyn_op_, dyn_state_, t, dt);
        // Update fault DOF data via wave_op + iterator (already wired in dyn_op).
        real_t V_max = ComputeVMaxFromDOFData(dyn_dof_data_);
        MPI_Allreduce(MPI_IN_PLACE, &V_max, 1, MPI_DOUBLE, MPI_MAX, comm_);
        if (V_max < V_deactivate_) {
            t_below_threshold_ += dt;
            if (t_below_threshold_ > exit_grace_t_) {
                HandoffDynamicToQD();
                dt = dt_qd_default_;
            }
        } else {
            t_below_threshold_ = 0;
        }
    }
}
```

The MPI_Allreduce-MAX for V_max is mandatory — without it ranks would diverge on regime decisions (the same issue as QD's RK45 error reduction).

4.7 Phase plan for cycle wiring

Phase	Goal	Depends on	Files created	Files modified	Acceptance
B.0	Reconcile geometry / mesh between QD and dynamic	none	(verification mesh bp5/mesh/hybrid_smoke_3km.msh if §11.2 picks new mesh)	none	Both seas_driver (BP5 QD) and the new seas_dynamic_driver (TPV102 mode) run a 50-step smoke on the same mesh and produce sensible (not necessarily reference) output

Phase	Goal	Depends on	Files created	Files modified	Acceptance
B.1	State-conversion helpers	A.6 (unified dynamic)	solver/seas_state_to_dynamic (functions: PackQDtoDOFData, PackQDtoDynamicBulk, ProjectCGToDG, ProjectDGToCG, ExtractFaultStateFromDOFData)	tools/bridge.{hpp,cpp}	Round-trip unit test: QD-state → DOFData + bulk → QD-state matches input to $\leq 1e-12$
B.2	SEASDynamicOperator polishing	A.6	solver/seas_dynamic_operator.{hpp,cpp} if not yet present (currently dynamic/seas_dynamic_operator.hpp is a thin wrapper; we promote it and add a Step() interface that owns the wave-RK4 + iterator wiring)	tools/system.{hpp,cpp}	Equivalent run via seas_dynamic_driver and via direct SEASDynamicOperator use produce same output
B.3	SEASHybridOperator skeleton + QD → dynamic handoff	B.1, B.2	solver/seas_hybrid_operator.{hpp,cpp} solver/regime_transfer.{hpp,cpp} drivers/seas_hybrid_driver.cpp	tools/runner.{hpp,cpp}	Run a synthetic test where QD is artificially forced into $V_{\max} > V_{\text{activate}}$ at $t = 1$ yr; verify the dynamic phase starts cleanly (no NaN, sensible σ field, $V_{\max}$ in dynamic phase comparable to a cold-start TPV102 with similar nucleation)

Phase	Goal	Depends on	Files created	Files modified	Acceptance
B.4	Dynamic → QD handoff + verification	B.3	(extend B.3 files)	none	Round-trip: QD → dynamic (synthetic trigger) → dynamic-quiet (V → 0) → QD; verify final ψ matches what plain QD would have produced over the same time interval to within tolerance documented in v2 (need to think about what tolerance is realistic — see §11.3)
B.5	End-to-end smoke	B.4	none	drivers/seas_hybrid polished for production	A 100-yr hybrid run on a small mesh detects a single (synthetic or natural) earthquake, switches regimes both ways, and finishes without crashing

Phase	Goal	Depends on	Files created	Files modified	Acceptance
B.6	First scientific cycle reproduction	B.5	(test infrastructure: cycle benchmark golden)	none	Reproduce the first earthquake from bp5_v62_fix_job7639398 (the 1593 yr golden) via hybrid mode within agreed tolerance — knowing this is a new benchmark since BP5 alone is QD-only and doesn't resolve the rupture, this is more of a sanity check than a true reference comparison

Phases B.0–B.4 are doable on local + small Frontera; B.5–B.6 require Frontera production allocation per feedback_frontera_approval.

5. Testing strategy

For Part A: - Per-phase regression: every phase must reproduce the relevant benchmark golden to byte-for-byte (or document a tolerance with explanation). - New unit tests for each strategy class: test_friction_law_strategy_rs.cpp, test_friction_law_strategy_lsw.cpp, test_state_evolution_strategy.cpp, test_nucleation_strategy.cpp. Each tests against a hand-computed numerical answer for a single QP. - Integration tests for SubStepIterator<> with each typedef pairing.

For Part B: - test_seas_state_bridge.cpp: round-trip QD ↔ DOFData ↔ bulk pack/unpack. - test_regime_transfer.cpp: the 7 tests sketched in dynamic_rupture_plan_v4.md §5.4 (TestQDtoFD-StressConsistency, TestQDtoFDSlipRateContinuity, TestFDtoQDRoundTrip, TestFDtoQDSlipAccumulation, TestFDtoQDEquilibrium, TestHybridSwitching, TestHybridFaultTipTaper). - test_hybrid_operator_smoke.cpp: a 100-step end-to-end run on a tiny inline mesh, asserting $dt > 0$, $V_{\max}$ bounded, regime switches happen at the configured thresholds.

6. Risk register (preliminary)

Risk	Detection	Mitigation
ψ drifts at handoff (different friction-law normalizations between QD and dynamic)	Round-trip test fails	Canonicalize the friction-law evaluation at one site (friction_coeff_stable.hpp); both sides call the same function.
Bulk u initialization discontinuity at QD $\rightarrow$ dynamic causes spurious wave packets that swamp the rupture signal	Visual inspection of $v(x, t = t_{\text{handoff}}^*)$ shows ringing inside the model	Equilibrium-correction sweep (§4.4 step 7) + fault-tip taper
Regime switching oscillates (V_{max} crosses thresholds repeatedly)	Step-trace shows alternating regimes	exit_grace_t_ parameter; require $V_{\text{max}} < V_{\text{deactivate}}$ for $\geq X$ seconds, default $X = 0.1$ s
Different per-rank V_{max} due to MPI rounding causes ranks to disagree on regime	Some ranks step the QD state, others step the dynamic state, system explodes	Always MPI_Allreduce(MAX) V_{max} before deciding; same pattern as QD's RK45 error reduction
Cycle test fails to reproduce known QD-only behavior in the inter-event phase	Long-time inter-event slip rate drifts vs golden	Verify QD phase byte-for-byte vs seas_driver standalone when hybrid is forced to QD-only via huge V_{activate}
Per-benchmark byte-for-byte regression in Part A migration phases	Regression script flags difference at $\geq 1e-12$	Migrate one benchmark at a time, compare exhaustively, fix per-axis until clean
"Dynamic settling" sweep washes out a real rupture	Hybrid-mode rupture has lower peak V than dynamic-only TPV102 with same nucleation	Make N_{settle} configurable; default cautiously to 5; offer a flag <code>--no-settle</code>

7. What's NOT in this plan

- TOML config migration (bp5_refactoring_plan v4 §12). v1 sticks to CLI flags. Deferring TOML lets us iterate on the framework shape first and design the config format around what we actually need.
- TPV104- and TPV205-style cycling. Both require nontrivial QD-side extensions (slip-law ψ , V_w field for TPV104; an LSW-equivalent QD operator that doesn't exist yet for TPV205).
- Adaptive mesh refinement during dynamic phase. Not envisioned; current MFEM AMR machinery is orthogonal to this plan.
- GPU port. dynamic_rupture_plan v4 already discusses GPU friction solver options; deferred to a separate plan.
- PML / SBI / ABC boundary work beyond what already exists in dynamic/pml_layer.{hpp,cpp}. dynamic_rupture_plan v4 Phase 2c proposed an SBI hybrid; that is its own project.
- Multi-physics extensions (poroelasticity, thermal pressurization) — outside the scope of v1.

8. Files summary (what gets touched in v1)

New (Part A)

```
dynamic/framework/friction_law_strategy.hpp
dynamic/framework/state_evolution_strategy.hpp
dynamic/framework/nucleation_strategy.hpp
dynamic/framework/substep_iterator.hpp
```

dynamic/framework/benchmarks.hpp
dynamic/framework/benchmark_config.hpp
dynamic/framework/run_time_loop.hpp
dynamic/framework/benchmarks/tpv102_config.{hpp,cpp}
dynamic/framework/benchmarks/tpv104_config.{hpp,cpp}
dynamic/framework/benchmarks/tpv205_config.{hpp,cpp}
drivers/seas_dynamic_driver.cpp
tests/unit/test_friction_law_strategy_rs.cpp
tests/unit/test_friction_law_strategy_lsw.cpp
tests/unit/test_state_evolution_strategy.cpp
tests/unit/test_nucleation_strategy.cpp
tests/unit/test_substep_iterator_unified.cpp

New (Part B)

solver/seas_state_bridge.{hpp,cpp}
solver/seas_hybrid_operator.{hpp,cpp}
solver/regime_transfer.{hpp,cpp}
drivers/seas_hybrid_driver.cpp
tests/unit/test_seas_state_bridge.cpp
tests/unit/test_regime_transfer.cpp
tests/integration/test_hybrid_operator_smoke.cpp
bp5/mesh/hybrid_smoke_3km.{geo,msh} (if §11.2 favors new mesh)

Modified (Part A)

drivers/tpv102_driver.cpp	– switch to TPV102Iterator; old code preserved as <code>_legacy</code> until A.7
drivers/tpv104_driver.cpp	– same
drivers/tpv205_driver.cpp	– same
Makefile, CMakeLists.txt	– add framework + test targets

Modified (Part B)

dynamic/seas_dynamic_operator.hpp	– promote to a real coupled operator with <code>Step()</code> (currently a thin pass through)
solver/seas_operator.hpp	– possibly add a <code>Step()</code> that hybrid op can call (currently the QD time)
drivers/seas_driver.cpp	– refactor <code>V_max</code> event detection out into the hybrid op (extract the ex
3/1e-6 code into a callable utility)	
Makefile, CMakeLists.txt	– add hybrid driver + tests

Removed (after A.7 only, gated on byte-for-byte goldens)

dynamic/tpv102_setup.hpp [tpv102_setup_total.hpp PRESERVED – still in C2 zone]
dynamic/tpv102_substep_iterator.{hpp,cpp}
dynamic/tpv102_friction_solver.hpp
dynamic/tpv102_nucleation.hpp
dynamic/tpv104_setup.hpp
dynamic/tpv104_substep_iterator.{hpp,cpp}
dynamic/tpv104_friction_solver.hpp
dynamic/tpv104_nucleation.hpp
dynamic/tpv205_setup.hpp
dynamic/tpv205_substep_iterator.{hpp,cpp}
dynamic/tpv205_friction.hpp
drivers/tpv102_driver.cpp [or rename `_legacy.cpp`]

drivers/tpv104_driver.cpp
drivers/tpv205_driver.cpp

9. Success criteria (exit conditions for v1 implementation)

1. Single binary `seas_dynamic_driver` reproduces TPV102, TPV104, TPV205 goldens to byte-for-byte (or to documented tolerance).
 2. Adding a fourth dynamic benchmark requires writing only:
 - One `MakeTPV{NEW}Config` factory function (~50 lines).
 - One `typedef` in `benchmarks.hpp`.
 - Optionally a new `FrictionLawStrategy<NewTag>` and/or `StateEvolutionStrategy<NewTag>` if the physics is genuinely new.
 - Zero changes to `wave_operator.{hpp,inl}`, `fault_face_flux.{hpp,cpp}`, `friction_solver.{hpp,cpp}`, or `godunov_flux.{hpp,cpp}`.
 3. Single binary `seas_hybrid_driver` runs a BP5-style aseismic phase, detects an earthquake, switches to dynamic, runs the rupture, switches back to QD, and continues — without operator intervention or restart.
 4. Hybrid-mode QD-only phase (when `V_activate = ∞`) produces the same time series as `seas_driver` standalone within rounding error.
 5. All goldens for TPV102/104/205 still reproduce after old-trio deletion.
 6. Round-trip QD ↔ dynamic ↔ QD with zero physical evolution preserves state to $\leq 1e-12$.
-

10. Approximate effort estimate

- Part A: ~3500 lines added (mostly framework + 3 small config factories), ~6000 lines deleted (after A.7). Net ~ -2500. Estimated 2–3 weeks of focused work plus 1 week of regression debugging.
 - Part B: ~1500 lines new code, 0 deleted. State-bridge + hybrid op + regime-transfer logic + driver + tests. Estimated 3–4 weeks plus Frontera time for B.5–B.6.
 - Total: 5–7 weeks for one developer working full-time, assuming the open questions in §11 are resolved without major redesign.
-

11. Open questions for round 2 (please weigh in)

These are decisions I'd rather not make unilaterally because they affect the framework shape:

11.1 Template vs virtual dispatch for the strategies

- Templates (current draft): zero runtime cost; cleaner inlining; but binary bloat ($\times 3$ inner loops).
- Virtual: cleaner driver (string-to-strategy at config time); minor runtime cost.
- Hybrid: templates internally, virtual at the top level (a `BenchmarkRunner` virtual base that dispatches into a concrete templated `SubStepIterator<>`).

My instinct: templates. Inner-loop perf matters for the wave op; binary bloat for 3–5 benchmarks is fine. But virtual is more conventional for a research code that's adding benchmarks frequently. Your call.

11.2 Mesh strategy for hybrid mode

- Reuse BP5 production mesh: validates against existing benchmark.
- Build a smaller hybrid-verification mesh: faster iteration.

- Both: production runs use BP5 mesh, smoke tests use a smaller box.

My instinct: both. A bp5/mesh/hybrid_smoke_3km.geo for fast iteration, BP5 production for real runs.

11.3 What's a realistic round-trip tolerance?

The test “QD → dynamic → QD with zero evolution preserves state to 1e-12” is what dynamic_rupture_plan v4 §5.4 proposed. But: - CG↔DG projection is exact only for polynomials of order $\leq p$; errors are $O(h^{p+1})$. - Equilibrium-correction sweep is dissipative. - Realistic round-trip preserves physics but not bits.

Better acceptance: $|\text{slip_final} - \text{slip_initial}| / |V_p \cdot \Delta t| < 1e-6$ over a controlled round-trip with no rupture. Open question — let's discuss.

11.4 TOML config: now or later?

bp5_refactoring_plan v4 §12 lays out a TOML-based driver. Doing TOML here would let us specify hybrid parameters cleanly (V_{activate} , exit_grace_t , mesh path, friction tag, etc.). But it's its own project (toml++ dependency, parser tests, validation logic) and easily 2 more weeks.

My instinct: defer to a separate plan. The CLI is fine for now; once we know what hybrid parameters actually matter, the TOML schema designs itself.

11.5 Dynamic rupture pure-mode driver: keep or sunset?

After seas_dynamic_driver is the unified entry point, the per-benchmark tpv1XX_driver.cpp files are redundant. feedback_no_local_reproducer says “don't run the TPV102 reproducer locally” — does that imply the per-benchmark driver is ceremonial and removable?

11.6 Ordering of Part A and Part B

Could be done in parallel: Part B doesn't depend on Part A's framework cleanup, only on having some dynamic operator that exposes a Step() method (which dynamic/seas_dynamic_operator.hpp already does as a thin wrapper). Doing Part A first makes Part B cleaner (one driver to switch into vs three); doing Part B first means we get the scientifically interesting result (cycles) sooner.

My instinct: Part A first. The framework cleanup makes Part B's SEASDynamicOperator use much more pleasant.

11.7 Other QD-state variables besides ψ

QD's RateStateFaultOperator is heavily templated; for NumSlipComp = 2, state is [s_dip, s_strike, ψ]. Are there any hidden QD state we need to migrate beyond ψ ? Specifically: does QD track any per-DOF accumulated quantity (cumulative slip in a different frame, integrated dissipation, etc.) that needs to bridge?

I checked RateStateFaultOperator and didn't see anything else, but a careful reread by someone closer to that code would be welcome.

12. Round-2 inputs we'd like

To turn this from a draft into an implementable plan, the next round should add:

1. Code-level review of §3.2 strategy interfaces against the existing per-benchmark substep iterators — find anywhere I've simplified away real complexity.
2. §11 answers (template-vs-virtual, mesh choice, tolerance, TOML, ordering, hidden QD state, driver sunset).

3. Concrete benchmark for the hybrid mode. Right now §4.7 phase B.6 is vague (“reproduce the first earthquake from bp5_v62_fix”). What’s the actual scientific test? Probably: run a 1.5–2 yr hybrid simulation, compare the first event’s slip distribution and recurrence time to literature (Kaneko et al. 2008, or whatever the SEAS BP5 reference is closest).
 4. A worked example of what the migrated TPV102 substep iterator looks like — a small prototype in a scratch branch. That would expose any abstractions I’ve waved at.
 5. CLI surface design — what flags `seas_dynamic_driver` takes (`--benchmark`, `--input-mesh`, `--output-dir`, ...) and what `seas_hybrid_driver` adds on top (`--V-activate`, `--V-deactivate`, `--exit-grace-time`, `--n-settle`, `--qd-checkpoint`, `--regime-log`).
-
-

13. Round-1 Adversarial Review (added 2026-04-29)

This section is the output of an adversarial review pass against v1. Findings are issues that should be resolved before this plan transitions to an implementation-ready v2. Severity, location, and a concrete fix are listed for each. IDs are stable — v2 should reference them when explaining how each issue was resolved.

Critical issues (must resolve before implementation)

[R-001] CRITICAL — §4.5 step 5.3: inverted inequality `V_deactivate << Vp` The plan reads:

with `V_qd` derived from the just-projected `u` (not from `V_final`, which is by definition `< V_deactivate << Vp`).

Both inequalities are wrong by $\sim 10^3$.

- `V_deactivate_default` is 1e-6 m/s (§4.3, §11 risk register).
- `Vp` on BP5 is 1e-9 m/s (3.17×10^{-2} m / yr $\approx 10^{-9}$ m/s; see `bp5/...` and `seas_driver.cpp`).

So `V_deactivate` $\gg$ `Vp` by three orders of magnitude. `V_final` at handoff is $\sim V_{deactivate}$, also $\gg Vp$. The motivating statement (`V_final` too small to use as a QD seed) is the opposite of reality: `V_final` at handoff is too large to be a QD steady-state seed (which sits near `Vp`), so we must back `V_qd` out from the projected `u` via the friction balance.

Fix:

```
- 3. Verify friction equilibrium  $|\tau_{total} - (\sigma_n \cdot f(V_{qd}, \psi_{final}, a) + \eta \cdot V_{qd})| / |\tau_{total}| < 1e-6$ 
-   with V_qd derived from the just-projected u (not from V_final, which is by definition
-   < V_deactivate << Vp).
+ 3. Verify friction equilibrium  $|\tau_{total} - (\sigma_n \cdot f(V_{qd}, \psi_{final}, a) + \eta \cdot V_{qd})| / |\tau_{total}| < 1e-6$ 
+   with V_qd derived from the just-projected u via the rate-state balance (not from V_final:
+   V_final  $\approx V_{deactivate} \approx 1e-6$  m/s  $\gg Vp \approx 1e-9$  m/s, so V_final is far above the QD
+   steady-state and would feed the next QD step a transient, not a relaxed inter-event state).
```

Implication for the test plan in §5: `test_regime_transfer.cpp::TestFDtoQDEquilibrium` must explicitly assert the back-out `V_qd` is within $\sim 10\times$ of `Vp`, not within $\sim 10\times$ of `V_final`.

[R-002] CRITICAL — §4.4 step 6: writes runtime QD traction into static-prestress slots `tau1_0`, `tau2_0`, `sigma_n0` The plan reads (verbatim):

```
data.tau1_0, data.tau2_0 = qd_tau_dip(i), qd_tau_strike(i) // background pre-stress
data.sigma_n_0 = qd_sigma_n(i)
```

This is conceptually wrong on two levels and would silently break the friction zero-point.

1. `tau1_0 / tau2_0 / sigma_n0` are STATIC background pre-stress slots, not running traction. From `dynamic/fault_face_flux.hpp:30-46`:

Background normal stress / Background shear pre-stress Distinct from `{sigma_n0, tau1_0, tau2_0}`: those carry STATIC background prestress and are zeroed under the total-Q dispatch contract

For TPV102, `dynamic/tpv102_setup.hpp:76-78`:

```
d.sigma_n0 = TPV102Params::sigma_n;    // 50 MPa
d.tau1_0    = 0.0;                      // no dip pre-stress
d.tau2_0    = TPV102Params::tau_ini;    // 75 MPa along-strike
```

These are constants over the run. Overwriting them with the QD-current traction at handoff changes the friction equation's reference and silently shifts `tau_strength` at every subsequent dynamic step.

2. Under the total-Q dispatch contract these slots are ZERO because the background pre-stress lives in bulk Q (per the comment block above). For benchmarks that use total-Q `EvaluateTotal / EvaluateADERTotal`, writing nonzero values into `tau*_0` double-counts the pre-stress.

The right place for time-varying contributions is the `tau*_nuc` slots, but at QD→dynamic handoff there is no nucleation pulse — the QD traction is already in the bulk Q via the projected `u`. So `tau*_nuc` should also be 0 (which the plan's comment “no nucleation in cycle mode” correctly states for `tau*_nuc`, just inconsistently with the surrounding `tau*_0` line).

Fix:

```
- data.tau1_0, data.tau2_0 = qd_tau_dip(i), qd_tau_strike(i) // background pre-stress
- data.sigma_n0 = qd_sigma_n(i)
+ // Background pre-stress slots: STATIC values from BenchmarkConfig (TPV102: tau2_0 = 75 MPa,
+ // sigma_n0 = 50 MPa, tau1_0 = 0). These DO NOT carry runtime QD traction – the QD-evolved
+ // traction is encoded in the bulk Q field (via the projected  $u \rightarrow \sigma$  in step 3) and the friction
+ // solver picks it up through tau*_trial, NOT through tau*_0.
+ data.tau1_0    = bench_cfg.tau1_0_default;           // typically 0
+ data.tau2_0    = bench_cfg.tau2_0_field(x_i, z_i);   // background only
+ data.sigma_n0  = bench_cfg.sigma_n0;
+
+ // For total-Q dispatch (TPV102 EvaluateADERTotal), these MUST be zeroed; pre-stress lives in Q.
+ // The bridge function MUST inspect the active EvaluateADER variant and zero these accordingly.
+ if (bench_cfg.uses_total_Q_dispatch) {
+   data.tau1_0 = data.tau2_0 = data.sigma_n0 = 0.0;
+ }
data.tau1_corr = data.tau2_corr = data.sigma_n_corr = 0;
data.tau1_nuc  = data.tau2_nuc  = data.sigma_n_nuc  = 0; // no nucleation in cycle mode
```

Test required: `test_regime_transfer.cpp::TestQDtoFDStressConsistency` must verify, on a synthetic handoff with known QD slip+loading, that `**tau_total = bulk_Q→trial + tau*_0**` matches the QD-side `tau_total` to $\leq 1e-10$ — not just that some total matches, but that the decomposition into background + dynamic parts is correct for both `Evaluate` and `EvaluateTotal` dispatch paths.

[R-003] CRITICAL — §4.3 design contradiction: `SEASHybridOperator : public TimeDependentOperator` with a `Mult` override is incompatible with “owns its own time-stepping loop” The class declaration in §4.3:

```
class SEASHybridOperator : public TimeDependentOperator {
...
    void Mult(const Vector& y, Vector& dy_dt) const override; // dispatches by regime
```

```
void Step(real_t& t, real_t& dt); // owns time-stepping + regime switching
};
```

Followed immediately by:

The hybrid op owns its own time-stepping loop (does not delegate to RK45 or the wave RK4 directly). This rules out exposing the hybrid op as a TimeDependentOperator to outside callers, but that's fine — the driver calls hybrid_op.Step(t, dt) in a loop.

These are mutually inconsistent:

1. TimeDependentOperator::Mult(y, dy_dt) requires a single fixed-size y. The hybrid op's logical state alternates between size $3 \cdot N_{\text{fault}}$ (QD) and size $9 \cdot N_{\text{dg}} + \text{sizeof}(D0FData) \cdot N_{\text{fault_qp}}$ (dynamic). A single contiguous Vector y either lies about the active regime's state or carries dead storage.
2. If Mult is implemented at all, MFEM ODESolvers will call it. Driving a regime switch from inside Mult creates re-entrancy (the underlying RK stage is mid-evaluation). The only way to make Mult safe is to forbid regime changes during a stage — but then regime changes happen between macro-steps anyway, where Step() already lives.

Fix — recommended: drop the TimeDependentOperator base entirely and make SEASHybridOperator a plain class with Step().

```
- class SEASHybridOperator : public TimeDependentOperator {
-     ...
-     void Mult(const Vector& y, Vector& dy_dt) const override; // dispatches by regime
-     void Step(real_t& t, real_t& dt); // owns time-stepping + regime switching
- };
+ class SEASHybridOperator {
+     ...
+     // Hybrid owns its own time loop – does NOT inherit TimeDependentOperator.
+     // QD sub-stepping uses the existing DormandPrinceRK45 driving qd_op_→Mult.
+     // Dynamic sub-stepping uses the existing wave-op RK / ADER driving dyn_op_.
+     // Regime switches happen between Step() calls, never inside a sub-stepper.
+     void Step(real_t& t, real_t& dt);
+ };
```

Update the prose immediately below the snippet to remove the “can't inherit cleanly” parenthetical (which becomes “doesn't inherit”). Update §6 “Modified files” to not claim hybrid op is a TimeDependentOperator subtype.

Test required: test_hybrid_operator_smoke.cpp must exercise Step() directly, not via ODESolver::Step(hybrid_op, ...).

[R-004] CRITICAL — §1.1 vs §4 / §8: **solver/** is in the no-edit zone but Part B adds files to it and proposes editing **solver/seas_operator.hpp** §1.1 lists:

No edits to bp5/, bp1/, bp2/, domain/, fault/, **solver/**, ...

But:

- §4.3 introduces solver/seas_hybrid_operator.{hpp,cpp} (new).
- §8 lists solver/seas_state_bridge.{hpp,cpp}, solver/seas_hybrid_operator.{hpp,cpp}, solver/regime_transfer.{hpp,cpp} as new files in solver/.
- §6 / §8 lists solver/seas_operator.hpp under Modified (Part B): “possibly add a Step() that hybrid op can call”.

Either the constraint or the location must move. The risk if left ambiguous is that the implementer either (a) violates the saved-feedback no-touch zone or (b) puts cycle-wiring code in a location that doesn't make architectural sense.

Fix — recommended: introduce a new sibling folder `solver/hybrid/` and put cycle-wiring code there. Restate §1.1 to clarify that existing files under `solver/` remain frozen but new sub-folders are allowed.

```
### 1.1 Hard "do not edit" zone
- - **No edits** to `bp5/`, `bp1/`, `bp2/`, `domain/`, `fault/`, `solver/`, `friction/dieterich_ruina.hpp`
+ - **No edits** to existing files under `bp5/`, `bp1/`, `bp2/`, `domain/`, `fault/`, `solver/`,
+   to `friction/dieterich_ruina.hpp`,
+   `friction/state_evolution.hpp`, `friction/slip_law_srw_psi.hpp`, `friction/friction_coeff_stable.hpp`,
+   or `dynamic/tpv102_setup_total.hpp`.
+ - New sub-folders are permitted: Part B adds files under `solver/hybrid/` (state bridge, hybrid op,
+   regime transfer); existing `solver/seas_operator.hpp`, `solver/time_stepper.hpp`,
+   `solver/seas_bdrload_operator.hpp` remain unchanged.
```

And in §6 / §8, move all `solver/seas_*` Part-B paths to `solver/hybrid/seas_*`.

For the proposed “modify `solver/seas_operator.hpp` to add `Step()`”: that violates the constraint. Replace it with an external utility:

```
- solver/seas_operator.hpp           - possibly add a Step() that hybrid op can call (currently the QD tim
+ solver/hybrid/qd_step_adapter.{hpp,cpp} - free functions that wrap an existing SEASQuasiDynamicOperator
+                                       in a "advance one macro step" API without modifying the
+                                       operator class itself
```

Similarly for §6's “drivers/seas_driver.cpp — refactor `V_max` event detection out”: §1.3 says additive only for `seas_driver.cpp`. Replace “refactor” with “leave untouched; the hybrid driver re-implements `V_max` detection by calling the same library helper that BP5 will optionally adopt later”:

```
- drivers/seas_driver.cpp           - refactor V_max event detection out into the hybrid op (extract the
+ drivers/seas_driver.cpp           - UNTOUCHED. Hybrid op replicates the V_max detection logic in
+                                       solver/hybrid/event_detector.hpp; BP5 may opt in later.
```

Moderate issues

[R-005] MODERATE — §4.4 step 4 option (b): “explicit Euler step backwards in time” is physics-meaningless

Solve a “kick-off” problem that sets v consistent with σ via one explicit Euler step backwards in time

For the elastodynamic system $\partial\sigma/\partial t = C : \nabla v$, $\rho \partial v/\partial t = \nabla \cdot \sigma$, “backwards in time from a stationary σ ” gives $v(t - dt) = v(t) - dt \cdot \nabla \cdot \sigma/\rho = -dt \cdot \nabla \cdot \sigma/\rho$ (since $v(t) = 0$ here), which is just a forward Euler step seeded from the $t=0$ stress with a sign flip. There's no causal sense in which this is “kick-off”; it's exactly equivalent to $v \approx +dt \cdot \nabla \cdot \sigma/\rho$ with the dt absorbed into a tunable amplitude, which is not what the literature does.

Fix: rewrite option (b) to either (i) drop it for $v1$ with a one-line note, or (ii) describe the standard approach: a single half-step velocity Verlet predictor, $v(t + dt/2) = (dt/2) \cdot \rho^{-1} \nabla \cdot \sigma(t)$. State whether the bulk strain ε that drives $\nabla \cdot \sigma$ matches the projected-from-QD u .

```
- - **Solve a "kick-off" problem** that sets v consistent with sigma via one explicit Euler step backwards in
+ - **Velocity-Verlet half-step seed**: set `v(t + dt/2) = (dt/2) * rho^-1 * nabla . sigma(t)` using the projected
+   stress, then run wave RK from that half-shifted state. Strictly more accurate than v=0 for the
+   first wave step but adds bulk-divergence plumbing. Defer to v2.
```


[R-006] MODERATE — §4.4 step 6: V_1, V_2 are not stored in QD state (“from latest RHS” understates the work) The plan instructs:

```
data.V1, data.V2 = qd_V_dip(i), qd_V_strike(i) // from latest RHS
```

But QD’s state vector is $[s_{\text{dip}}, s_{\text{strike}}, \psi]$ per fault DOF (§1.2, confirmed in `fault/rate_state_fault.hpp`). V is computed inside every `Mult` call from the friction balance (Brent solve) using current ψ + current traction; it is not persistently stored anywhere accessible by name after `Mult` returns.

If the bridge runs at handoff time t , the latest stored RK45 stage may be at $t + a_k \cdot dt$ for some Butcher-tableau $a_k \neq 0$, not at t . So either:

1. The bridge re-evaluates the friction balance at time t using the stored $(slip, \psi)$ and a freshly-solved traction. This is the right answer but is not “from latest RHS”.
2. The QD operator caches V_{last} at the end of every `Mult`, exposed via an accessor. This is invasive (touches `solver/seas_operator.hpp`, see [R-004]).

Fix: spell out that the bridge does its own friction-balance solve.

```
- data.V1, data.V2 = qd_V_dip(i), qd_V_strike(i) // from latest RHS
+ // Re-solve the rate-state friction balance in the bridge using:
+ //    $\psi$  from qd_state_, slip from qd_state_, traction from the just-rerun elasticity solve (step 2).
+ // The bridge MUST NOT trust an in-flight RK stage's  $V$  – those are at  $t + a_k \cdot dt$ , not at the
+ // handoff  $t$ . Use the same Brent driver as RateStateFaultOperator::Mult (call into
+ // friction/dieterich_ruina.hpp::SolveSlipRateBrent).
+ (data.V1, data.V2) = SolveBalanceForV(qd_state_psi(i), qd_traction(i), bench_cfg);
```

Add a unit test `test_seas_state_bridge.cpp::TestVConsistencyAtHandoff` that pins this within $1e-12$ of an independent Brent solve at the same (ψ, τ, σ_n) .

[R-007] MODERATE — §3.2.3 `NucleationStrategy` interface omits the overwrite/accumulate distinction §3.1 explicitly identifies the two semantics:

TPV102: Gaussian `smoothStep`, overwrite τ_{2_0} per macro-step TPV104: Gaussian `smoothStep`, accumulate $\Delta\tau_{\text{nuc}}$ per substep

But the strategy `Apply(...)` signature in §3.2.3 takes $(t_{\text{macro}}, dt_{\text{sub}}, sub_step_index)$ and gives no contract for whether it should overwrite or accumulate. The implementer reading §3.2.3 alone has to guess.

Furthermore, §3.1 conflates two different write targets: TPV102 writes to `tau2_0` (the static-prestress slot), while TPV104 writes to `tau*_nuc` (the time-varying nucleation slot). These are different `DOFData` fields with different read paths inside `EvaluateADER` vs `EvaluateADERTotal` (`dynamic/fault_face_flux.hpp:288–348`). A strategy that doesn’t make the target slot explicit will silently land in the wrong field.

Fix: declare the contract in the strategy header.

```
template <class Tag>
struct NucleationStrategy {
+ // Contract:
+ //   - Apply() is called exactly once per substep with sub_step_index  $\in [0, num\_sub\_steps)$ .
+ //   - The implementation declares which DOFData slot it writes to via a static constexpr:
+ //       static constexpr DOFDataSlot target_slot = ...;
+ //   valid values: PreStressTau2_0, NucPerturbationTau2Nuc, NoOp.
+ //   - For overwrite-per-macro-step semantics, the impl checks (sub_step_index == 0) and
+ //     writes the smoothStep value to target_slot; subsequent substeps are no-ops.
+ //   - For accumulate-per-substep semantics, the impl writes the per-substep increment to
+ //     target_slot at every call; macro-step start is detected via sub_step_index == 0
+ //     reset of the accumulator.
```

```

+ // - Static / no-op strategies write nothing and target_slot = NoOp.
static void Apply(
    std::vector<DOFData>& dof_data,
    const Vector& fault_x, const Vector& fault_z,
    const NucleationParams& params,
    real_t t_macro,
    real_t dt_sub,
    int sub_step_index);
};

```

Add test_nucleation_strategy.cpp cases per tag: TPV102 should write tau2_0 only on sub_step_index == 0, TPV104 should accumulate tau2_nuc over a sequence of substeps and produce the same total as the old per-driver code at end-of-macro-step, TPV205 should be a no-op for t > 0.

[R-008] MODERATE — §4.4 step 3: CG→DG projection cost claim (“element-local, no MPI, basis-change matrix per element”) assumes equal polynomial orders. The QD CG H^1 space and the dynamic DG L^2 space are not guaranteed to share a polynomial order. The plan’s BP5 QD code uses a configurable order (config/bp5_params.hpp), and the dynamic side uses its own (dynamic/wave_operator.hpp). If these differ, CG → DG is not a basis change — it’s a real L^2 projection per element with a non-trivial mass matrix (still element-local, but a real solve, not a multiplication by a precomputed matrix).

The plan’s cited 0.1 s for BP5 size depends on this assumption.

Fix: make the assumption explicit and gate it.

```

3. **Project u from CG  $H^1$  space (QD) to DG  $L^2$  space (dynamic).** This is element-local
- (basis-change matrix per element), no MPI. Dynamic_rupture_plan v4 §5.2 estimates
- ~0.1 s for BP5 size.
+ (no MPI). If the QD CG order p_qd matches the dynamic DG order p_dyn, this reduces to
+ a precomputed per-element basis-change matrix multiplication (~0.1 s for BP5 size,
+ per dynamic_rupture_plan v4 §5.2). If p_qd ≠ p_dyn, this is a per-element  $L^2$  projection
+ via the local DG mass matrix – still element-local but ~10× more expensive. v1 ASSERTS
+ p_qd == p_dyn at hybrid-op construction time; supporting unequal orders is deferred.

```

[R-009] MODERATE — §9 success criterion #6 vs §11.3: 1e-12 round-trip tolerance is contradicted within the same plan. §9.6 states: “Round-trip QD ↔ dynamic ↔ QD with zero physical evolution preserves state to $\leq 1e-12$.”

§11.3 states: “Realistic round-trip preserves physics but not bits ... $|\text{slip}_{\text{final}} - \text{slip}_{\text{initial}}| / |\text{Vp} \cdot \Delta t| < 1e-6$ over a controlled round-trip.”

These cannot both be acceptance gates. Implementer will reach §9 first, write a 1e-12 test, fail, then fix the test (or fudge it) before reading §11.3.

Fix: delete the 1e-12 line from §9 and link to §11.3’s eventual decision. Until §11.3 is resolved, success criterion #6 is “TBD pending §11.3”.

```

- 6. Round-trip QD ↔ dynamic ↔ QD with zero physical evolution preserves state to  $\leq 1e-12$ .
+ 6. Round-trip QD ↔ dynamic ↔ QD tolerance: see §11.3 (open question). v1 implementation
+ must include the round-trip test but the numeric tolerance is set in v2. Provisional:
+  $|\text{slip}_{\text{final}} - \text{slip}_{\text{initial}}| / (\text{Vp} \cdot \Delta t_{\text{round-trip}}) < 1e-6$ .

```

[R-010] MODERATE — §4.6 trigger logic: `dyn_stepper.Step(*dyn_op_, dyn_state_, t, dt)` does not bind `dyn_dof_data_`. The dynamic state is not `dyn_state_` alone — it is the pair (`dyn_state_`, `dyn_dof_data_`) (per §4.3 storage list and confirmed by `WaveOperator + FaultFaceFlux` interaction in `dynamic/wave_operator.inl`, `fault_face_flux.cpp`). The wave RK / ADER driver advances both: bulk Q via the wave op, fault-DOF state via the iterator.

The pseudocode line as written would advance only the bulk Q, leaving `DofData` frozen.

Fix:

```
-     dyn_stepper_.Step(*dyn_op_, dyn_state_, t, dt);
-     // Update fault DOF data via wave_op + iterator (already wired in dyn_op).
+     // dyn_op_ owns both dyn_state_ AND dyn_dof_data_; its Step() advances the bulk Q
+     // (wave RK / ADER) AND fault DofData (substep iterator) in lockstep, as the existing
+     // tpv102/104/205 drivers already do.
+     dyn_op_>Step(t, dt);
```

Also update §4.3's class definition to make `dyn_op_>Step(t, dt)` the canonical entry point (and remove `Mult` from `SEASDynamicOperator` if present, per [R-003]'s pattern).

[R-011] MODERATE — §4.7 phase B.0 acceptance: TPV102 on the BP5 mesh with TPV102 parameters won't produce sensible behavior. Acceptance reads:

Both `seas_driver` (BP5 QD) and the new `seas_dynamic_driver` (TPV102 mode) run a 50-step smoke on the same mesh and produce sensible (not necessarily reference) output

TPV102 parameters (depth profile, $\sigma_n = 50$ MPa uniform, $\tau_{ini} = 75$ MPa pure strike-slip, fault z extent ± 15 km, Gaussian nucleation in (x, z)) are tied to the TPV102 SCEC mesh's geometry. Loading TPV102 parameters onto the BP5 production mesh (which has a 60 km $\times$ 40 km fault, depth-dependent a/b, etc.) gives garbage — V profiles will lock everywhere, or run away depending on which (a – b) intersection happens by accident.

Fix: rephrase the acceptance to use separate parameter sets per mesh, or explicitly construct a “TPV102-on-BP5-mesh” hybrid parameter set as part of B.0.

```
- Acceptance: Both `seas_driver` (BP5 QD) and the new `seas_dynamic_driver` (TPV102 mode) run a
- 50-step smoke on the same mesh and produce sensible (not necessarily reference) output
+ Acceptance: `seas_driver` runs a 50-step smoke on bp5_3km.msh with BP5 params, `seas_dynamic_driver`
+ runs a 50-step smoke on its own TPV102 box mesh with TPV102 params; both finish without NaN and
+ produce monotonic-in-time fault output. The "same mesh" goal is deferred to B.3, by which point
+ a hybrid-compatible parameter set (BP5-style depth profile on the smaller verification mesh) is
+ defined.
```

[R-012] MODERATE — §3.2.2 `SlipLawSRW` strategy depends on `data.V_w` but the plan does not specify how `V_w` is populated through the framework. §3.2.2's `SlipLawSRWPsiTag` impl says:

```
static void Advance(DofData& d, real_t V_abs, real_t dt_sub);
// Calls UpdateStateAnalyticSlipLawSRW; reads d.V_w
```

But:

1. `DofData` (per `fault_face_flux.hpp`:25–85) does not declare a `V_w` field.
2. TPV104 currently passes `V_w` via a side-channel array (per §2.1: “`V_w` side-channel (TPV104 only)”).
3. §3.3's `BenchmarkConfig` has `Vw_field` as a `std::function`, but that's a startup-time field-evaluator, not a per-DOF stored value.

So the framework as drafted does not actually wire V_w through to the strategy.

Fix: add V_w to `DOFData` (this is an append, allowed under §1.2’s “DOFData field offsets — only append new fields”) and populate from `BenchmarkConfig::Vw_field` in `InitializeFaultDOFsFromConfig`.

```
struct DOFData
{
    ...
    real_t lsw_d_c = 0.0;
+   real_t V_w      = 0.0;    ///< Strong slip-rate weakening  $V_w$  (TPV104 slip-law-SRW); 0 elsewhere.
};
```

Note this is the only `DOFData` append called for in v1 — flag it explicitly so the implementer doesn’t append silently in multiple phases.

Low / quality issues

- [R-013] §3.5 phase A.0: “regression script” is named but never specified. Add a one-paragraph spec — what tool diffs the goldens, what the tolerance is, where the script lives. Without this, A.3-A.5 acceptance is subjective.
- [R-014] §3.5 phase A.5: requires “Frontera $np > 1$ ” but per saved feedback `feedback_local_mpi_up_to_10`, local $np \leq 10$ is acceptable for smoke runs. Soften to “Frontera-only above $np = 10$; $np \in [2, 10]$ may be local”.
- [R-015] §3.4 driver dispatch: `ParseBenchmarkFlag(argc, argv)` is called before `MakeTPV1XXConfig(argc, argv)` and the latter parses its own flags. Spell out that `ParseBenchmarkFlag` is a non-consuming peek; a flag-parsing library that consumes `--benchmark` would break the per-config parsers.
- [R-016] §5 testing list typo: “TestHybridFault tipTaper” → “TestHybridFaultTipTaper”.
- [R-017] §4.4 step 1 layout note: “straight copy from `qd_state_`” — clarify that `qd_state_` is interleaved AOS as $[s1, s2, \psi]$ per node; the bridge must respect the layout (not assume SOA blocks).
- [R-018] §4.5 step 5.5 (final substep): “set `qd_state_ = (slip_final,  $\psi_{\text{final}}$ )`” — note that this respects the same $[s1, s2, \psi]$ interleaving as [R-017] (consistency check, easy to get wrong on opposite ends of the cycle).

Summary of required v2 actions

Severity	Count	Must-fix-before-impl	Required test
CRITICAL	4 (R-001...R-004)	yes	TestFDtoQDEquilibrium, TestQDtoFDStressConsistency, test_hybrid_operator_smoke, no-test (policy)
MODERATE	8 (R-005...R-012)	strongly recommended	TestVConsistency-AtHandoff, test_nucleation_strategy_per_tag, projection-order assertion, etc.
LOW	6 (R-013...R-018)	nice-to-have	none required

Verdict: PASS WITH FIXES. The architecture sketch (§3 strategies, §4 hybrid op skeleton, §3.5 / §4.7 phase plans) is sound. The four CRITICAL items are conceptual or scope errors that will silently corrupt

results or violate constraints if implemented as written; v2 should resolve them before any code lands. The MODERATE items mostly under-specify interfaces and would generate avoidable rework.

End of v1 draft. Hand-off note for v2: the architecture sketch is the load-bearing part; everything else (line counts, file lists, phase ordering) is negotiable.